

# **UNIVERSIDAD PRIVADA DOMINGO SAVIO**

## **FACULTAD DE INGENIERIA CARRERA DE INGENIERIA EN SISTEMAS**



## **PRE-DEFENSA - PROYECTO HOTEL REAL**

Universitarios:

Miguel Alemán

Denilson Condori

Jose Acosta Morales

Jonathan Zubieta Mendoza

Docente:

Ing. Marianela Lourdes Salas Flores

La Paz, 21 De Septiembre De 2026

## TABLA DE CONTENIDO

|                                                                              |    |
|------------------------------------------------------------------------------|----|
| 1. INTRODUCCIÓN .....                                                        | 1  |
| 2. OBJETIVOS .....                                                           | 1  |
| 2.1 OBJETIVO GENERAL .....                                                   | 1  |
| 2.2 OBJETIVOS ESPECÍFICOS .....                                              | 1  |
| 3. JUSTIFICACIÓN .....                                                       | 2  |
| 4. MARCO TEÓRICO .....                                                       | 2  |
| 4.1 INTRODUCCIÓN A LOS SISTEMAS OPERATIVOS GNU/LINUX PARA SERVIDOR .....     | 2  |
| 4.1.1 Qué es un sistema operativo: .....                                     | 2  |
| 4.1.2 Características de los Sistemas Operativos: .....                      | 2  |
| 4.1.3 Clasificación de los Sistemas Operativos: .....                        | 3  |
| 4.1.4 Funcionamiento de los Sistemas Operativos: .....                       | 3  |
| 4.1.5 Versiones de Linux (Distribuciones): .....                             | 3  |
| 4.2 CARACTERÍSTICAS Y SERVICIOS EN GNU/LINUX .....                           | 3  |
| 4.3 GESTIÓN DE USUARIOS Y FICHEROS .....                                     | 3  |
| 4.4 SEGURIDAD Y VIRTUALIZACIÓN .....                                         | 4  |
| 4.5 REQUISITOS TÉCNICOS Y HERRAMIENTAS UTILIZADAS .....                      | 4  |
| 5. MARCO PRÁCTICO .....                                                      | 4  |
| 5.1 IMPLEMENTACIÓN Y ADMINISTRACIÓN DE UN SERVIDOR LINUX .....               | 4  |
| 5.1.2 DISEÑO DE BASE DE DATOS (DIAGRAMA ER) .....                            | 5  |
| 5.1.3 DIAGNÓSTICO Y PLANIFICACIÓN .....                                      | 6  |
| 5.2 DESARROLLO DE LA ADMINISTRACIÓN DE UN SERVIDOR LINUX (PASO A PASO) ..... | 6  |
| 5.2.2 ADMINISTRACIÓN DE USUARIOS Y GRUPOS .....                              | 7  |
| 5.2.3 SERVICIO SSH Y FIREWALL (UFW) .....                                    | 7  |
| 5.2.4 CONFIGURACIÓN DEL MOTOR DE BASE DE DATOS .....                         | 9  |
| 5.2.5 DESPLIEGUE DEL SERVIDOR DNS (BIND9) .....                              | 10 |
| 5.2.6 DESPLIEGUE DEL ENTORNO WEB (BACKEND Y FRONTEND) .....                  | 14 |
| 6. CONCLUSIONES Y RECOMENDACIONES .....                                      | 15 |
| 8. BIBLIOGRAFÍA .....                                                        | 16 |
| 9. ANEXOS .....                                                              | 16 |

## 1. INTRODUCCIÓN

El presente proyecto tiene como finalidad el diseño, implementación y administración de un entorno de red completo basado en sistemas operativos GNU/Linux, aplicado al desarrollo de un sistema informático transaccional para el "Hotel Real", ubicado en la ciudad de La Paz, Bolivia.

Para lograr un entorno que simule un escenario de producción empresarial, el proyecto ha sido desarrollado por un equipo multidisciplinario de 4 personas. Esta división de trabajo permitió abordar de manera simultánea la arquitectura de red, la seguridad perimetral, la administración de la base de datos y la codificación de los aplicativos Frontend y Backend.

El alcance del avance actual (50%) demuestra la correcta interconexión entre el ecosistema Linux (con servicios críticos como SSH, DNS, UFW, y PostgreSQL operando en un entorno de red Host-Only aislado y seguro) y una plataforma web de hotelería que permite a los usuarios pre-visualizar habitaciones, registrarse y realizar simulaciones de reserva mediante transferencias bancarias (QR).

## 2. OBJETIVOS

### 2.1 OBJETIVO GENERAL

Implementar y administrar un servidor virtual bajo el entorno GNU/Linux (Ubuntu 24.04 LTS) para gestionar servicios de red esenciales (DNS, Web, Base de Datos, Firewall), soportando el despliegue de una plataforma de reservas hoteleras desarrollada por un equipo de 4 integrantes, garantizando la seguridad, disponibilidad y el manejo adecuado de archivos físicos.

### 2.2 OBJETIVOS ESPECÍFICOS

- **Diagnóstico y Virtualización:** Desplegar una máquina virtual en VMware con configuración de doble adaptador (NAT y Host-Only) para garantizar conectividad ininterrumpida independiente del proveedor de internet físico.
- **Arquitectura de Red:** Diseñar la infraestructura lógica estableciendo una IP Estática (192.168.19.10) y resolución de nombres locales mediante BIND9 (hotelreal.com.bo).
- **Seguridad y Permisos:** Administrar usuarios y grupos específicos (hotel\_staff), implementando políticas estrictas de sistema de archivos jerárquico (chmod 770 y 700) para salvaguardar los comprobantes de pago de los clientes.
- **Despliegue de Servicios:** Instalar y configurar servicios críticos, incluyendo SSH para administración remota, UFW como cortafuegos perimetral, y PostgreSQL 16 como motor de base de datos relacional.

- **Desarrollo de Software Integrado:** Desarrollar una API RESTful monolítica (Backend) y una interfaz de usuario responsiva (Frontend) que interactúen en tiempo real con el servidor Linux.
- **Trabajo Colaborativo:** Distribuir las cargas de trabajo en un equipo de 4 personas, para la unificación del código fuente en el entorno de despliegue.

### 3. JUSTIFICACIÓN

En el sector hotelero actual, la digitalización de reservas es fundamental. Sin embargo, desarrollar únicamente una página web sin comprender la infraestructura de red subyacente deja al sistema vulnerable a ataques informáticos y caídas de servicio.

La implementación de este servidor Linux en un entorno controlado justifica la necesidad de formar profesionales integrales que comprendan el ciclo de vida completo del software. Al involucrar a 4 personas en el proyecto, se simula el entorno real de una empresa tecnológica donde convergen el Administrador de Sistemas (Sysadmin), el Administrador de Base de Datos (DBA), el Desarrollador Backend y el Desarrollador Frontend.

Asimismo, el uso de herramientas de código abierto (Open Source) como Ubuntu, PostgreSQL y Python, justifica la viabilidad económica del proyecto, demostrando que es posible montar una infraestructura de clase mundial con cero costo de licenciamiento.

### 4. MARCO TEÓRICO

#### 4.1 INTRODUCCIÓN A LOS SISTEMAS OPERATIVOS GNU/LINUX PARA SERVIDOR

##### 4.1.1 Qué es un sistema operativo:

Un Sistema Operativo (SO) es el software base que actúa como intermediario entre el hardware del computador y los programas de usuario. Es el gestor maestro de los recursos del sistema (CPU, memoria, almacenamiento y periféricos). En entornos empresariales, el SO debe estar optimizado para procesar múltiples peticiones de red simultáneas de forma eficiente sin requerir interfaces gráficas pesadas.

##### 4.1.2 Características de los Sistemas Operativos:

- **Concurrencia:** Capacidad de ejecutar múltiples procesos (como el servidor DNS y la Base de datos) al mismo tiempo mediante la asignación de tiempos de CPU.
- **Seguridad y Aislamiento:** Mecanismos como el *Ring* de protección del procesador, que evitan que un proceso de usuario colapse el núcleo del sistema.

- **Gestión de Red:** Implementación de la pila de protocolos TCP/IP a nivel de Kernel.
- **Abstracción de Hardware:** Provee una interfaz genérica (drivers) para que el software no deba preocuparse por el hardware subyacente.

#### 4.1.3 Clasificación de los Sistemas Operativos:

Se pueden clasificar por su arquitectura interna (Monolíticos como Linux, o Microkernels), por el número de usuarios simultáneos (Multiusuario), y por su propósito comercial (Desktop, Mobile, y Servidor). El enfoque de este proyecto radica puramente en la clasificación de **Servidor**, donde prima la estabilidad 24/7 sobre la estética visual.

#### 4.1.4 Funcionamiento de los Sistemas Operativos:

El núcleo (Kernel) recibe llamadas al sistema (*System Calls*) desde el espacio de usuario. En nuestro proyecto, cuando Python desea leer un comprobante QR, realiza una petición al Kernel de Linux, el cual verifica los permisos del archivo mediante el sistema VFS (Virtual File System) antes de entregar los datos al disco duro físico.

#### 4.1.5 Versiones de Linux (Distribuciones):

A diferencia de Windows, Linux posee el Kernel abierto, permitiendo la creación de "Distribuciones". Las más prominentes para servidores incluyen:

- **Red Hat Enterprise Linux (RHEL):** Orientada a corporaciones con soporte pago.
- **CentOS / AlmaLinux:** Clones comunitarios de RHEL.
- **Ubuntu Server:** Desarrollada por Canonical, es la distribución elegida para este proyecto debido a su inmensa comunidad, repositorios actualizados (APT) y excelente documentación para despliegues web modernos.

### 4.2 CARACTERÍSTICAS Y SERVICIOS EN GNU/LINUX

GNU/Linux es el estándar de facto en la infraestructura web mundial. Sus servicios funcionan a través de "Demonios" (Daemons), que son programas que se ejecutan en segundo plano administrados por **Systemd**. Estos servicios son modulares; si el servicio Web falla, no compromete al servicio DNS o a la base de datos.

### 4.3 GESTIÓN DE USUARIOS Y FICHEROS

El modelo de seguridad de Linux asigna a cada archivo un Propietario, un Grupo y "Otros". Los permisos se calculan de forma octal (Ej: 770 o 644). Este modelo es la piedra angular del

proyecto, permitiendo que la carpeta `/var/hotel_reportes` sea invisible para usuarios comunes, pero accesible para el grupo `hotel_staff`.

#### 4.4 SEGURIDAD Y VIRTUALIZACIÓN

La seguridad perimetral se delegó a **UFW (Uncomplicated Firewall)**, una interfaz simplificada para iptables en Linux que rechaza paquetes TCP/UDP no deseados. La virtualización mediante VMware permitió simular un switch virtualizado (Red Host-Only) garantizando que el entorno no sufra colisiones de IP (Conflictos ARP) con la red externa de la universidad o el hogar.

#### 4.5 REQUISITOS TÉCNICOS Y HERRAMIENTAS UTILIZADAS

El equipo de 4 integrantes consolidó el uso de las siguientes tecnologías y requisitos:

- **Infraestructura:** Hipervisor VMware Workstation.
- **Requisitos Mínimos Asignados a la VM:** 2 vCPUs, 2 GB de Memoria RAM, 20 GB de Almacenamiento Dinámico, y 2 adaptadores de red (NAT + Host-Only).
- **Sistema Operativo:** Ubuntu Server 24.04 LTS.
- **Base de Datos:** PostgreSQL 16 (Relacional, cumple con ACID y 3FN).
- **Backend (Lógica de Negocios):** Python 3.12 utilizando el framework *FastAPI* y el servidor ASGI *Uvicorn*, seleccionado por su alto rendimiento y manejo asíncrono.
- **Frontend (Vista Cliente/Admin):** HTML5, JavaScript Vainilla, y el framework *Tailwind* CSS servido bajo arquitectura SPA (Single Page Application) ligera.
- **Herramientas de Red Linux:** Netplan, BIND9 (DNS), UFW.

### 5. MARCO PRÁCTICO

#### 5.1 IMPLEMENTACIÓN Y ADMINISTRACIÓN DE UN SERVIDOR LINUX

##### 5.1.1 ARQUITECTURA PROPUESTA

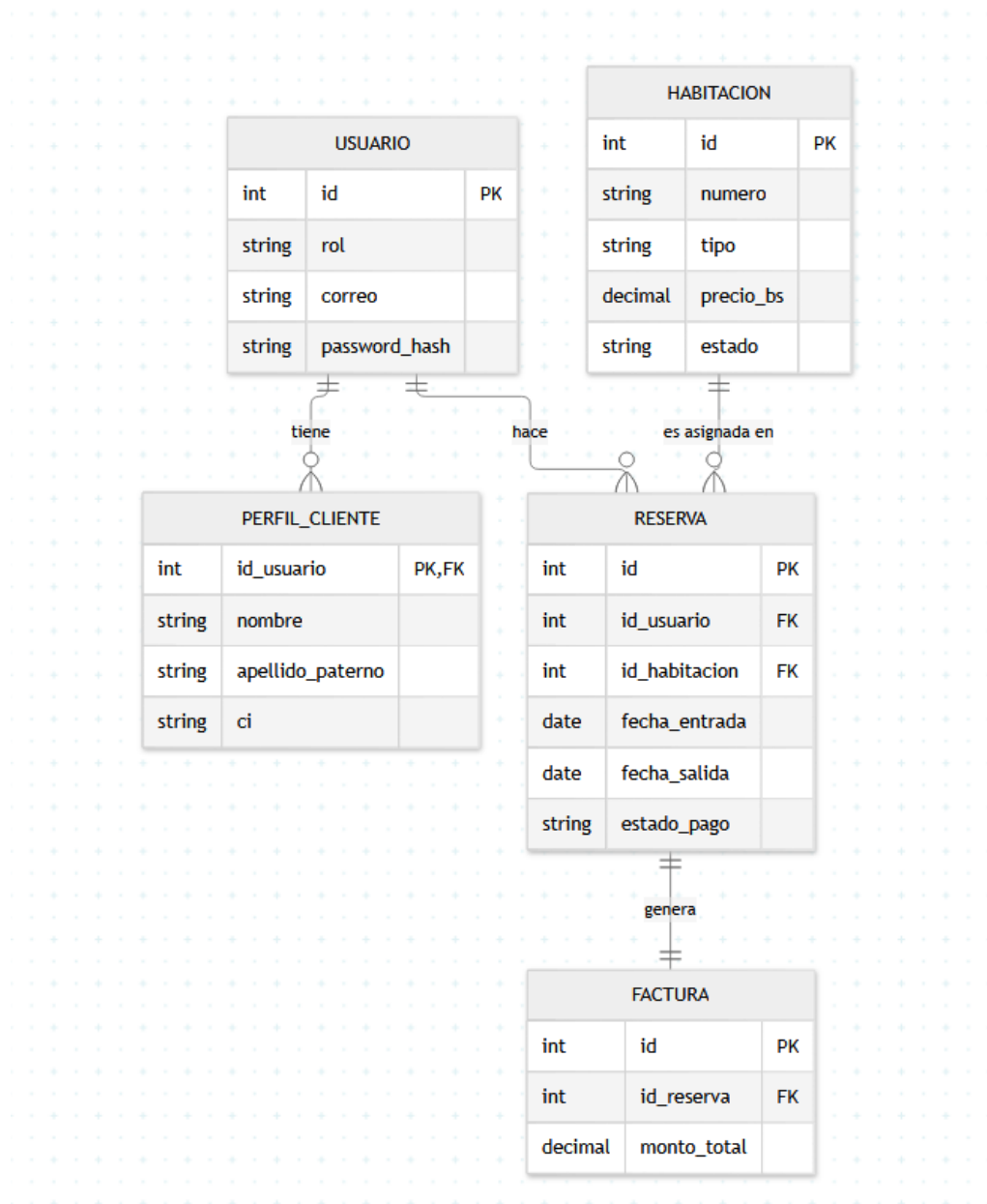
La arquitectura del proyecto se divide en el servidor central (Ubuntu 24.04 LTS) alojado en VMware, y los clientes que acceden desde el sistema anfitrión. El servidor hospeda el motor de base de datos PostgreSQL, el servicio DNS BIND9, y el aplicativo desarrollado por el equipo de 4 personas, compuesto por un Backend en Python (FastAPI) y un Frontend en HTML/TailwindCSS.

### 5.1.2 DISEÑO DE BASE DE DATOS (DIAGRAMA ER)

Para sostener la aplicación, se diseñó la base de datos db\_hotelreal normalizada en 3FN.

Figura 1

Diagrama Entidad-Relación de la Base de Datos del Hotel



Nota. El diagrama muestra las 5 tablas principales, identificando sus llaves primarias (PK) y foráneas (FK), desarrollado para aislar credenciales de datos personales.

### 5.1.3 DIAGNÓSTICO Y PLANIFICACIÓN

Figura 2

Entorno de Virtualización Inicial

```
user@srv-web:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:30:c2:2b brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    inet 192.168.10.20/24 brd 192.168.10.255 scope global noprefixroute ens33
        valid_lft forever preferred_lft forever
    inet 192.168.70.128/24 brd 192.168.70.255 scope global dynamic noprefixroute ens33
        valid_lft 1391sec preferred_lft 1391sec
    inet6 fe80::20c:29ff:fe30:c22b/64 scope link
        valid_lft forever preferred_lft forever
3: ens37: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:30:c2:35 brd ff:ff:ff:ff:ff:ff
    altname enp2s5
    inet 192.168.19.128/24 brd 192.168.19.255 scope global dynamic noprefixroute ens37
        valid_lft 1390sec preferred_lft 1390sec
    inet6 fe80::2f6b:fe5b:c2bf:bb50/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
user@srv-web:~$ ^C
user@srv-web:~$
```

Nota. Diagnóstico inicial de la red en VMware antes de las configuraciones estáticas.

## 5.2 DESARROLLO DE LA ADMINISTRACIÓN DE UN SERVIDOR LINUX (PASO A PASO)

### 5.2.1 CONFIGURACIÓN DE RED ESTÁTICA

Se procedió a configurar Netplan (/etc/netplan/00-installer-config.yaml) asignando la IP 192.168.19.10 al adaptador Host-Only para evitar cortes de red.

Figura 3

Validación de IP Estática

```
user@srv-web: ~ 76x15
user@srv-web:~$ sudo nano /etc/netplan/00-installer-config.yaml
```

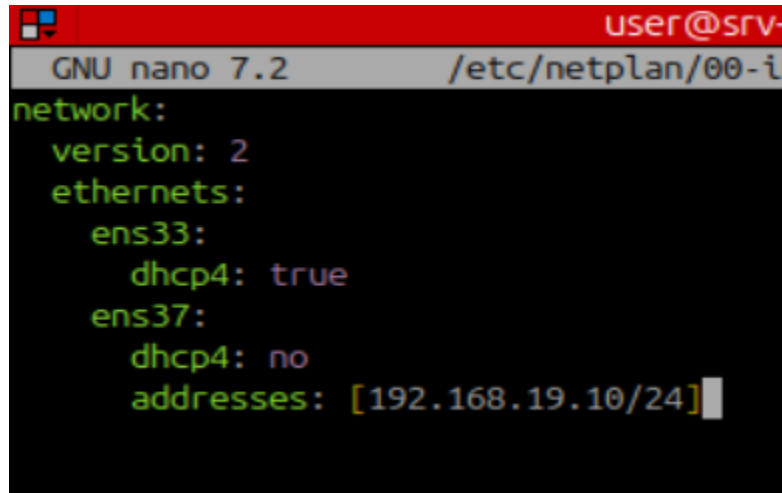
Nota. Comando 'ip a' confirmando la asignación de la dirección IPv4.

### 5.2.2 ADMINISTRACIÓN DE USUARIOS Y GRUPOS

Se crearon los usuarios de administración y el grupo hotel\_staff.

Figura 4

Creación de Usuarios

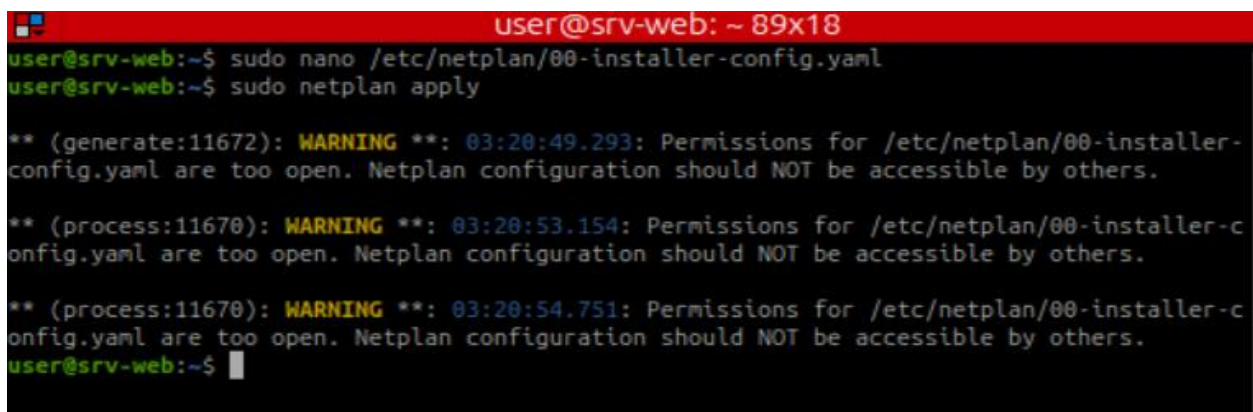


```
user@srv-  
GNU nano 7.2 /etc/netplan/00-i  
network:  
  version: 2  
  ethernet:  
    ens33:  
      dhcp4: true  
    ens37:  
      dhcp4: no  
      addresses: [192.168.19.10/24]
```

Nota. Comandos adduser para integrar a la recepción y gerencia al grupo de trabajo.

Figura 5

Asignación de Permisos de Carpeta



```
user@srv-web: ~ 89x18  
user@srv-web:~$ sudo nano /etc/netplan/00-installer-config.yaml  
user@srv-web:~$ sudo netplan apply  
  
** (generate:11672): WARNING **: 03:20:49.293: Permissions for /etc/netplan/00-installer-  
config.yaml are too open. Netplan configuration should NOT be accessible by others.  
  
** (process:11670): WARNING **: 03:20:53.154: Permissions for /etc/netplan/00-installer-c  
onfig.yaml are too open. Netplan configuration should NOT be accessible by others.  
  
** (process:11670): WARNING **: 03:20:54.751: Permissions for /etc/netplan/00-installer-c  
onfig.yaml are too open. Netplan configuration should NOT be accessible by others.  
user@srv-web:~$
```

Nota. Ejecución de chmod 770 y chown sobre /var/hotel\_reportes.

### 5.2.3 SERVICIO SSH Y FIREWALL (UFW)

Para la administración remota, se habilitó SSH y se aseguró el servidor cerrando puertos no utilizados.

Figura 6

Instalación de Servicios Base

```
user@srv-web:~$ sudo chmod 000 /etc/netplan/00-installer-config.yaml
user@srv-web:~$
```

Nota. Instalación de paquetes iniciales del sistema.

Figura 7

Activación del Firewall

```
user@srv-web:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UP0DOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:0c:29:38:c2:2b brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    inet 192.168.18.28/24 brd 192.168.18.255 scope global noprefixroute ens33
        valid_lft forever preferred_lft forever
    inet 192.168.78.128/24 brd 192.168.78.255 scope global dynamic noprefixroute ens33
        valid_lft 1731sec preferred_lft 1731sec
    inet6 fe80::28c:29ff:fe38:c22b/64 scope link
        valid_lft forever preferred_lft forever
3: ens37: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:0c:29:38:c2:35 brd ff:ff:ff:ff:ff:ff
    altname enp2s3
    inet 192.168.19.18/24 brd 192.168.19.255 scope global noprefixroute ens37
        valid_lft forever preferred_lft forever
    inet6 fe80::28c:29ff:fe38:c235/64 scope link
        valid_lft forever preferred_lft forever
```

Nota. Ejecución de ufw enable tras permitir el puerto 22.

Figura 8

Estado del Firewall Perimetral

```
user@srv-web:~$ sudo apt update && sudo apt install ufw -y
sudo ufw allow 22/tcp # SSH
sudo ufw allow 80/tcp # Web
sudo ufw allow 53/tcp # DNS
sudo ufw allow 53/udp # DNS
sudo ufw --force enable
```

Nota. Comprobación de reglas activas de UFW.

Figura 9

Conexión SSH Exitosa

```
user@srv-web:~$ sudo ufw status verbose
Estado: activo
Acceso: on (low)
Predeterminado: deny (entrantes), allow (salientes), disabled (enrutados)
Perfiles nuevos: skip

Hasta                Acción            Desde
-----
22/tcp               ALLOW IN         Anywhere
80/tcp               ALLOW IN         Anywhere
53/tcp               ALLOW IN         Anywhere
53/udp               ALLOW IN         Anywhere
22/tcp (v6)          ALLOW IN         Anywhere (v6)
80/tcp (v6)          ALLOW IN         Anywhere (v6)
53/tcp (v6)          ALLOW IN         Anywhere (v6)
53/udp (v6)          ALLOW IN         Anywhere (v6)

user@srv-web:~$
```

Nota. Ingreso seguro al servidor mediante SSH desde el cliente Windows.

## 5.2.4 CONFIGURACIÓN DEL MOTOR DE BASE DE DATOS

Se instaló PostgreSQL 16 para el almacenamiento persistente.

Figura 10

Instalación de PostgreSQL

```
user@srv-web:~$ sudo groupadd hotel_staff
sudo useradd -m -s /bin/bash recepcion
sudo passwd recepcion
Nueva contraseña:
Vuelva a escribir la nueva contraseña:
passwd: contraseña actualizada correctamente
user@srv-web:~$ sudo useradd -m -s /bin/bash gerencia
sudo passwd gerencia
Nueva contraseña:
CONTRASEÑA INCORRECTA: De alguna manera, en la contraseña se lee el nombre del usuario
Vuelva a escribir la nueva contraseña:
Las contraseñas no coinciden.
Nueva contraseña:
Vuelva a escribir la nueva contraseña:
Las contraseñas no coinciden.
Nueva contraseña:
Vuelva a escribir la nueva contraseña:
passwd: contraseña actualizada correctamente
user@srv-web:~$ sudo usermod -aG hotel_staff recepcion
sudo usermod -aG hotel_staff gerencia
user@srv-web:~$ sudo mkdir -p /var/hotel_reportes
sudo chown root:hotel_staff /var/hotel_reportes
sudo chmod 770 /var/hotel_reportes
user@srv-web:~$
```

Nota. Despliegue de los binarios de postgresql-contrib.

Figura 11

Creación de Base de Datos db\_hotelreal

```
C:\Windows\System32>ssh user@192.168.19.10
user@192.168.19.10's password:
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 7.0.0-31-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

El mantenimiento de seguridad expandido para Applications está desactivado

Se pueden aplicar 105 actualizaciones de forma inmediata.
13 de estas son actualizaciones de seguridad estándares.
Para ver estas actualizaciones adicionales, ejecute: apt list --upgradable

3 actualizaciones de seguridad adicionales se pueden aplicar con ESM Apps.
Aprenda más sobre cómo activar el servicio ESM Apps at https://ubuntu.com/esm

*** System restart required ***

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

user@srv-web:~$
```

Nota. Ingreso al shell psql para la creación de la base de datos principal.

Figura 12

Configuración de Usuarios de BD

```
user@srv-web:~$ sudo apt install postgresql postgresql-contrib -y
sudo systemctl enable postgresql
sudo systemctl start postgresql
```

Nota. Creación del usuario admin\_db\_hotel con su respectiva contraseña.

### 5.2.5 DESPLIEGUE DEL SERVIDOR DNS (BIND9)

Para traducir hotelreal.com.bo a nuestra IP local.

Figura 13

Instalación de BIND9

```
user@srv-web:~$ sudo mkdir -p /var/soporte_hotel
sudo chown postgres:postgres /var/soporte_hotel
sudo chmod 700 /var/soporte_hotel
user@srv-web:~$
```

Nota. Proceso de instalación del servidor de nombres.

Figura 14

Configuración del named.conf.local

```
user@srv-web:~$ sudo -u postgres psql
psql (16.15 (Ubuntu 16.15-0ubuntu0.24.04.1))
Type "help" for help.

postgres=# ALTER USER postgres PASSWORD 'admin123';
CREATE USER admin_db_hotel WITH PASSWORD 'hotel123';
CREATE DATABASE db_hotelreal OWNER admin_db_hotel;
\q
ALTER ROLE
CREATE ROLE
CREATE DATABASE
user@srv-web:~$
```

Nota. Declaración de la zona maestra hotelreal.com.bo.

Figura 15

Archivo de Zona Directa

```
user@srv-web:~$ sudo systemctl status postgresql
● postgresql.service - PostgreSQL RDBMS
   Loaded: loaded (/usr/lib/systemd/system/postgresql.service; enabled; preset: e
   Active: active (exited) since Mon 2026-09-21 03:42:12 -04; 5min ago
   Main PID: 14547 (code=exited, status=0/SUCCESS)
   CPU: 31ms

sep 21 03:42:11 srv-web systemd[1]: Starting postgresql.service - PostgreSQL RDBMS.
sep 21 03:42:12 srv-web systemd[1]: Finished postgresql.service - PostgreSQL RDBMS.
lines 1-8/8 (END)
```

Nota. Configuración del archivo db.hotelreal vinculando dominios a la IP 192.168.19.10.

Figura 16

Chequeo de Configuración

```
user@srv-web:~$ ls -ld /var/soporte_hotel
drwx----- 2 postgres postgres 4096 sep 21 03:44 /var/soporte_hotel
user@srv-web:~$
```

Nota. Uso de named-checkconf para validar la sintaxis sin errores.

Figura 17

Resolución de Conflictos DNS 1

```
user@srv-web:~$ sudo apt install bind9 bind9utils -y
sudo systemctl enable bind9
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
Se instalarán los siguientes paquetes adicionales:
  bind9-utils
Paquetes sugeridos:
  bind-doc
Se instalarán los siguientes paquetes NUEVOS:
  bind9 bind9-utils bind9utils
```

Nota. Errores de sintaxis detectados inicialmente.

Figura 18

Resolución de Conflictos DNS 2

```
user@srv-web:~$ sudo nano /etc/bind/named.conf.local
```

Nota. Corrección de los registros NS para solucionar el error.

Figura 19

Reinicio de Servicios DNS

```
GNU nano 7.2 /etc/bind/named.conf.local
//
// Do any local configuration here
//
// Consider adding the 1918 zones here, if they are not used in your
// organization
//include "/etc/bind/zones.rfc1918";
zone "hotelreal.com.bo" {
    type master;
    file "/etc/bind/db.hotelreal";
};
```

Nota. Systemctl restart named para aplicar las zonas directas.

Figura 20

Prueba de Nslookup Exitosa

```
user@srv-web: ~ 81x21
GNU nano 7.2 /etc/bind/db.hotelreal *
$TTL      604800
@          IN      SOA      ns.hotelreal.com.bo. admin.hotelreal.com.bo. (
                                2      ; Serial
                                604800  ; Refresh
                                86400   ; Retry
                                2419200 ; Expire
                                604800  ) ; Negative Cache TTL
;
@          IN      NS       ns.hotelreal.com.bo.
@          IN      A        192.168.19.10
www        IN      A        192.168.19.10
```

Nota. El sistema resolvió correctamente hotelreal.com.bo.

Figura 21

Análisis de Logs DNS

```
user@srv-web:~$ sudo systemctl restart bind9
```

Nota. Revisión de advertencias menores en BIND9.

Figura 22

Corrección de Permisos de Ficheros

```
user@srv-web:~$ ping -c 4 www.hotelreal.com.bo
ping: www.hotelreal.com.bo: Nombre o servicio desconocido
```

Nota. Aplicación de chmod 644 a db.hotelreal para permitir lectura.

### 5.2.6 DESPLIEGUE DEL ENTORNO WEB (BACKEND Y FRONTEND)

El equipo desarrolló un Backend robusto en Python (FastAPI) para gestionar las conexiones a BD y un Frontend dinámico servido en el puerto 80.

Figura 23

Preparación del Código Fuente

```
GNU nano 7.2 /etc/bind/db.hotelreal
@ TTL 004800
@ IN SOA ms.hotelreal.com.bo. admin.hotelreal.com.bo. (
; Serial
; Refresh
; Retry
; Expire
; Negative Cache TTL
;
@ IN NS ms.hotelreal.com.bo.
@ IN A 192.168.19.10
ms IN A 192.168.19.10
www IN A 192.168.19.10
```

Nota. Sincronización e inicialización del código web en el servidor Linux.

Figura 24

Levantamiento del Backend Python (Uvicorn)

```
user@srv-web:~$ nslookup www.hotelreal.com.bo 127.0.0.1
;; Got SERVFAIL reply from 127.0.0.1
Server: 127.0.0.1
Address: 127.0.0.1#53

** server can't find www.hotelreal.com.bo: SERVFAIL

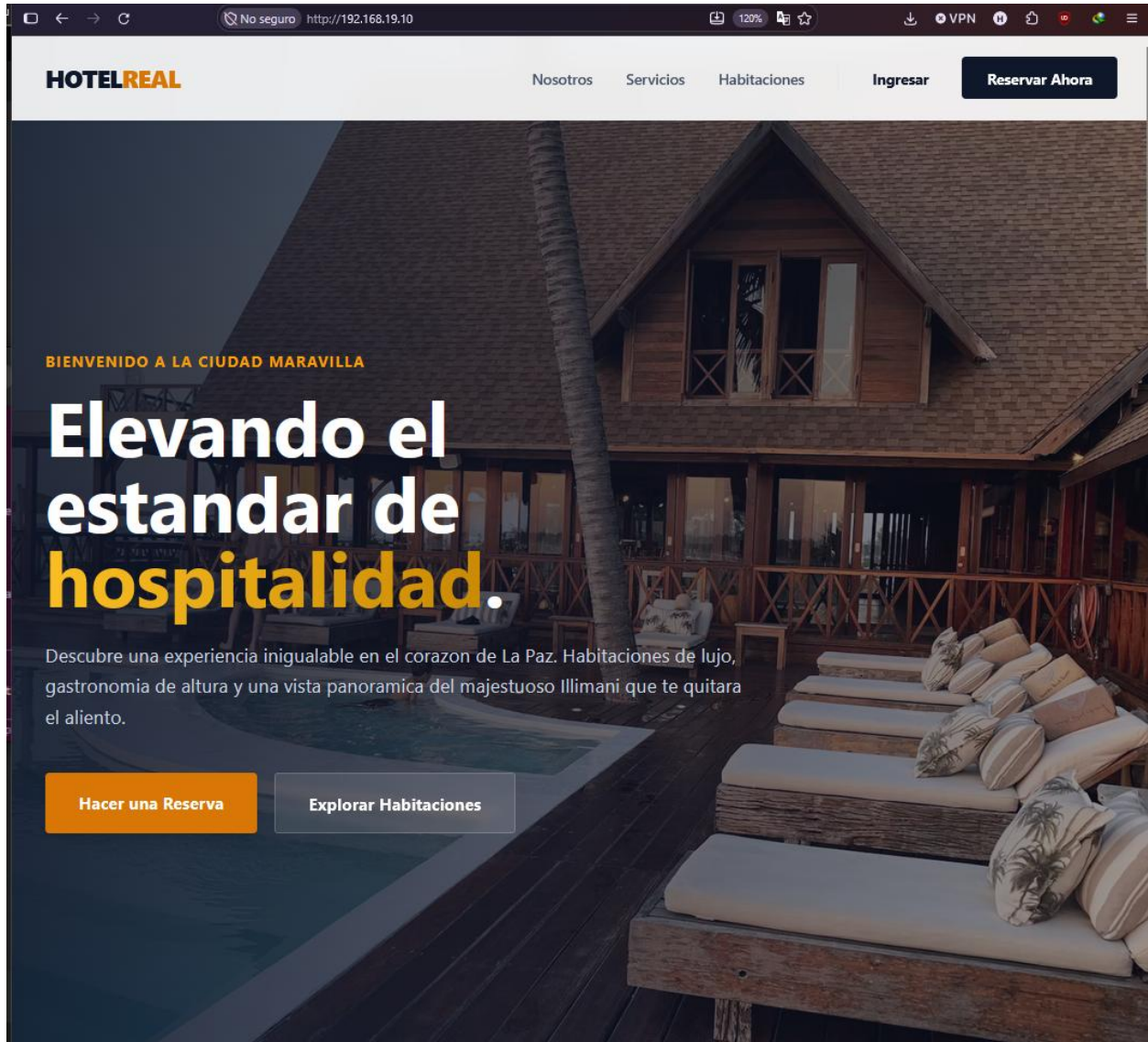
user@srv-web:~$ sudo named-checkconf
user@srv-web:~$ sudo named-checkzone hotelreal.com.bo /etc/bind/db.hotelreal
zone hotelreal.com.bo/IN: loaded serial 2
OK
user@srv-web:~$ sudo chmod 644 /etc/bind/db.hotelreal
user@srv-web:~$ sudo systemctl restart named
user@srv-web:~$ nslookup www.hotelreal.com.bo 127.0.0.1
Server: 127.0.0.1
Address: 127.0.0.1#53

Name: www.hotelreal.com.bo
Address: 192.168.19.10
user@srv-web:~$
```

Nota. Servidor ASGI Uvicorn iniciado en el puerto 8000 esperando peticiones.

Figura 25

Interfaz Frontend de Hotel Real



Nota. Renderizado final de la aplicación web conectada exitosamente a la base de datos tras otorgar permisos GRANT PRIVILEGES en PostgreSQL y abrir el puerto 8000 en UFW.

## 6. CONCLUSIONES Y RECOMENDACIONES

### CONCLUSIONES

Se concluye que el despliegue de un sistema de reservas hoteleras sobre una infraestructura GNU/Linux es totalmente viable y altamente seguro cuando se configuran los permisos adecuados (3FN en Base de Datos, UFW en red, y jerarquía de permisos de archivo). El trabajo

conjunto de 4 integrantes permitió abordar simultáneamente la complejidad del Backend (Python FastAPI), Frontend (HTML/Tailwind), Administración de Base de Datos y la Seguridad de Red. Además, se comprobó que los errores de conectividad (como SERVFAIL o bloqueos de puerto) pueden resolverse mediante el análisis de logs y la correcta aplicación de comandos (chmod, chown, y GRANT PRIVILEGES).

## RECOMENDACIONES

Para la segunda fase del proyecto, enfocada en alcanzar el 100% de la integración, se recomienda encarecidamente:

Crear un demonio (Systemd) que garantice el auto-arranque del aplicativo hotelero en caso de que el servidor físico se reinicie.

Automatizar la creación de copias de seguridad de PostgreSQL usando tareas programadas (Crontab).

Implementar un servidor de correo interno (Postfix) que permita el envío de notificaciones de las reservas aprobadas al cliente.

## 8. BIBLIOGRAFÍA

Canonical Ltd. (2024). *Ubuntu Server 24.04 LTS Official Documentation*. Recuperado de [ubuntu.com/server/docs](https://ubuntu.com/server/docs)

The PostgreSQL Global Development Group. (2024). *PostgreSQL 16 Documentation*. Recuperado de [postgresql.org/docs/16/](https://postgresql.org/docs/16/)

Internet Systems Consortium. (2024). *BIND 9 Administrator Reference Manual*.

Tiangolo, S. (2024). *FastAPI Framework Documentation*. Recuperado de [fastapi.tiangolo.com](https://fastapi.tiangolo.com)

## 9. ANEXOS

### ANEXO 1: Archivo de Configuración de Red Estática (Netplan)

Ruta: /etc/netplan/00-installer-config.yaml

YAML

```
network:
  version: 2
  ethernets:
    ens33:
      dhcp4: true
    ens37:
      dhcp4: no
      addresses: [192.168.19.10/24]
```

## ANEXO 2: Configuración del Servidor DNS (BIND9)

Ruta: /etc/bind/named.conf.local

Plaintext

```
zone "hotelreal.com.bo" {
    type master;
    file "/etc/bind/db.hotelreal";
};
```

Ruta: /etc/bind/db.hotelreal (Zona Directa)

Plaintext

```
$TTL      604800
@         IN      SOA      ns.hotelreal.com.bo. admin.hotelreal.com.bo. (
                                2           ; Serial
                                604800      ; Refresh
                                86400       ; Retry
                                2419200     ; Expire
                                604800 )    ; Negative Cache TTL
;
@         IN      NS       ns.hotelreal.com.bo.
@         IN      A        192.168.19.10
ns        IN      A        192.168.19.10
www       IN      A        192.168.19.10
```

## ANEXO 3: Modelo Relacional (Data Definition Language) - PostgreSQL

Ruta: /home/user/proyecto\_hotel/database/schema\_v2.sql

SQL

```
CREATE TABLE usuario (
    id SERIAL PRIMARY KEY,
    rol VARCHAR(20) NOT NULL CHECK (rol IN ('admin', 'cliente')),
    correo VARCHAR(100) UNIQUE NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    fecha_registro TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE perfil_cliente (
    id_usuario INTEGER PRIMARY KEY REFERENCES usuario(id),
    nombre VARCHAR(100) NOT NULL,
    apellido_paterno VARCHAR(100) NOT NULL,
    apellido_materno VARCHAR(100),
    ci VARCHAR(20) UNIQUE NOT NULL,
    telefono VARCHAR(20),
    nit VARCHAR(20),
    razon_social VARCHAR(100)
);

CREATE TABLE habitacion (
    id SERIAL PRIMARY KEY,
    numero VARCHAR(10) UNIQUE NOT NULL,
    piso INTEGER NOT NULL,
    tipo VARCHAR(50) NOT NULL,
    vista VARCHAR(50) DEFAULT 'Interior' CHECK (vista IN ('Interior', 'A la Calle', 'Al Illimani')),
    precio_bs DECIMAL(10, 2) NOT NULL,
```

```

        estado VARCHAR(20) DEFAULT 'disponible' CHECK (estado IN ('disponible', 'ocupada', 'mantenimiento'))
    );

CREATE TABLE reserva (
    id SERIAL PRIMARY KEY,
    id_usuario INTEGER REFERENCES usuario(id),
    id_habitacion INTEGER REFERENCES habitacion(id),
    fecha_entrada DATE NOT NULL,
    fecha_salida DATE NOT NULL,
    estado_pago VARCHAR(20) DEFAULT 'pendiente' CHECK (estado_pago IN ('pendiente', 'verificando',
'pagado', 'rechazado')),
    ruta_comprobante VARCHAR(255)
);

CREATE TABLE factura (
    id SERIAL PRIMARY KEY,
    id_reserva INTEGER REFERENCES reserva(id),
    nit_emision VARCHAR(20) NOT NULL,
    razon_social_emision VARCHAR(100) NOT NULL,
    monto_total DECIMAL(10, 2) NOT NULL,
    fecha_emision TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

```